# Closing Implementation Gaps Against Kissat: Nine Repairs, Four Refutations, and a Correction to How We Measure

Tactix development report for Andrew

4 September 2026

## 1 Objective and outcome

The objective is to close every identified implementation gap against Kissat: every case where Tactix performs the same work and takes longer. Being within 10 % on Tier D Ranks 1–40 is the floor, not the finish. B28+C5 is the regression gate and N1–N4 are preserved.

**Outcome.** Ranks 1–40 moved from 8,781.1 s to **8,535 s**, which is **1.112**× Kissat's 7,675.1 s, and B28+C5 from 0.495 to **0.463**. Nine repairs landed, each gated 40/40 valid with 10,000 fuzz cases and zero failures on both legs. Every one is behaviour-preserving: solver counters are identical between arms on every instance measured, and on all forty gate ranks. None of them changes the search.

The net per-rule gap against Kissat fell from roughly 500 s to **428 s**.

## 2 What was repaired

**A diagnostic that shipped.** BVE ran an occurrence audit on every candidate elimination: both polarities' entire occurrence streams walked, every large clause dereferenced, each scanned twice, a literal vector built per clause. Its only consumers are two trace calls that are empty inline functions in a production build. Compiling it out is worth **170.6 s** across the tier, the largest single repair of the project.

**Factor, at provably matched work.** Kissat charges 733 M factor ticks on Rank 22 and Tactix 764 M in the same currency, for 8.9 s against 25.2 s. Per scanned entry Tactix took two random cache misses where Kissat takes one, because Kissat's skip mark is a clause header bit read from the line it already loads. Moving both marks into the header, which then made the occurrence identifier dead and let the watch record fall from 16 bytes to 8, plus replacing the remainder pool's map of bucket vectors with an open-addressed table, took Factor from 281.8 s to 238 s tier-wide.

**Congruence's doubled ITE gates.** Kissat canonicalises an ITE triple before storing a gate, so one entry covers a relation and its output-negated twin. Tactix keyed on the sequence as found and kept both. Rank 22's gate count falls from 10.48 M to 7.10 M against Kissat's 6.91 M, the merge count is unchanged at 1,543,122, and every counter is identical.

**Four quadratic removals.** Substitution retires a literal from every clause holding it, so its occurrence list is consumed whole; removing entries one clause at a time scanned 40 G elements on Rank 22. The root-satisfied binary scans recounted each pair through the watch list. The binary rewrite's duplicate test rescanned the representative's prefix per image. And removing a literal searched the large list and the ordered stream separately for the same reference. Each is now one pass.

## 3 What was refuted

Four attempts measured neutral or unsound and were dropped rather than committed: accumulating Factor's scan charge in a local, guarding fourteen trace-only computations that the compiler already eliminates, hoisting BVE's per-clause vectors, and bulk-clearing an eliminated variable's binaries, whose precondition fails in the rule's own tests.

Two negative results matter as much as the repairs. Propagation is 42 % of runtime and looked 1.5× slower on Rank 31, but that instance runs 2.7× Kissat's conflicts and so carries longer watch lists; at equal propagation counts on Rank 22 the rates are within 9 %. And Sweep, which an earlier report named the largest remaining gap on the strength of one rank, is 120 s *faster* than Kissat across the tier.

## 4 A correction to how we measure

Two measurement errors were found and fixed, and both changed conclusions.

The hand-written A/B campaigns ran at four CPUs and 16 GiB, which the benchmark runner documents as the *debug* contract; the gates use twelve and 48 GiB. A bandwidth-sensitive result read at the wrong allocation is not evidence. The two-word Factor watch record was refuted under those conditions and, retried correctly after the cache-miss confound was removed, proved worth 2.8 s on Rank 22.

More consequentially, the tier total's noise band is far wider than the 50 s this project had assumed. The final repair measured a consistent −1.4 s on Rank 22 across interleaved repeats and then returned a gate **72.8 s slower** with all forty conflict counts identical to the previous gate. The change cannot add work, so that is machine variation. The tier total is a validity check and a coarse trend; it cannot arbitrate a behaviour-identical change. Paired same-node repeats must.

## 5 Where the remaining gap is

| rule | Tactix | Kissat | gap | ratio |
|---|---:|---:|---:|---:|
| Substitute | 282.2 s | 169.6 s | +112.6 | 1.66 |
| Factor | 237.8 s | 129.8 s | +108.1 | 1.83 |
| Vivify | 680.4 s | 576.6 s | +103.8 | 1.18 |
| Strengthen | 112.4 s | 36.1 s | +76.2 | 3.11 |
| Congruence | 318.7 s | 245.8 s | +72.9 | 1.30 |
| BVE | 260.0 s | 194.1 s | +65.9 | 1.34 |
| TransitiveReduce | 49.3 s | 34.4 s | +14.9 | 1.43 |
| Backbone | 66.5 s | 69.4 s | −2.9 | 0.96 |
| Sweep | 441.4 s | 564.9 s | −123.6 | 0.78 |

Read Strengthen's row with care: Kissat runs forward subsumption inside `eliminate`, not in its `subsume` phase, so the row overstates the gap. The fair pairing is BVE plus Strengthen against eliminate plus subsume, 372 s against 230 s. Tactix does 12 % *fewer* forward steps and only 8 % more eliminations, so that 142 s is unit cost and it is the largest single gap pairing left.

Searching for more instances of the ITE pattern, a Tactix work counter exceeding Kissat's for the same outcome, found none in BVE or Strengthen. The remaining gaps are diffuse per-visit cost rather than surplus work, with one architectural exception: during Sweep, Tactix maintains both the sparse watch lists and the dense occurrence index, where Kissat's sweep has only the latter. Removing that duplication needs a dense propagation path Tactix does not have.

## 6 Limitations

Rank 31 remains the worst instance at 671 s against 262 s, and its gap is search trajectory rather than implementation: it takes 2.7× the conflicts. That is a separate problem and it is not closed.

Two documentation budgets are over: `PROGRESS.md` at 3,773 lines against 350 and `docs/implementation-notes.md` at 591 against 560. Compaction was not requested.

Evidence is in `benchmarks/reports/2026-09-02-unit-cost-and-fidelity-stack.md`, sections 3.12 to 3.21, with campaigns `5010` through `5054`.